
University of Michigan–Ann Arbor

Department of Electrical Engineering and Computer Science

EECS 475 Introduction to Cryptography, Fall 2024

Instructor: Ke Wu

GSI: Forest Zhang

Homework 3

Student: Shikha Nukala (desigirl)

Collaborators (if any): N/A

Please refer to the course information document for important collaboration policies. Among other policies, you must list all collaborators and, moreover, cite any external resources that you have used for each exercise. You must write all solutions on your own and may not share this material outside class.

Due: Oct 10, 2024 (at 11:59 PM US Eastern Time)

Exercise 1

(20 points) Let $F: \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ be pseudorandom function. Show that each of the following MACs is insecure, even if used to authenticate fixed-length messages. (In each case Gen outputs a uniform $k \in \{0, 1\}^n$, n being even, and $\langle i \rangle$ denotes an $n/2$ -bit binary encoding of the integer i).

- (a) To authenticate a message $m = m_1 \| \dots \| m_\ell$, where $m_i \in \{0, 1\}^n$, compute $t := F_k(m_1) \oplus \dots \oplus F_k(m_\ell)$.
- (b) To authenticate a message $m = m_1 \| \dots \| m_\ell$, where $m_i \in \{0, 1\}^{n/2}$, compute $t := F_k(\langle 1 \rangle \| m_1) \oplus \dots \oplus F_k(\langle \ell \rangle \| m_\ell)$.

Solution

(a) First part.

In this part the MAC scheme to authenticate message $m = m_1 \| \dots \| m_\ell$, where $m_i \in \{0, 1\}^n$ by computing $t := F_k(m_1) \oplus \dots \oplus F_k(m_\ell)$ is insecure because we can perform a forgery attack using the $\oplus$ in the tag computation. If we have adversary A , the adversary knows the MAC tag for the message $m = m_1 \| \dots \| m_\ell$ because the tag is not secret from the adversary. With the *plus* operand in the tag computation, the adversary can use another message $m_{reordered} = m_2 \| m_1 \| \dots \| m_\ell$, the tag is

computed $t_{reordered} := F_k(m_2) \oplus F_k(m_1) \oplus \dots \oplus F_k(m_\ell)$. Since the $\oplus$ operation is still performed between the same fixed n bits of each part of the message, it holds that $t = t_{reordered}$. Therefore if A sends message m 's bits reordered to the tag generator, the tag output validates the original message without the message being the original. Therefore this MAC is vulnerable to forgery attacks even with fixed-length messages and is insecure.

(b) Second part.

The difference from the last part of the question is that this MAC uses message sequence lengths $n/2$. In this case we have a message $m = m_1 \parallel \dots \parallel m_\ell$ where $m_i \in \{0, 1\}^{n/2}$. The tag generation algorithm encodes the number of the block of the $n/2$ bits part of the message along with the partial message. We can break this security by computing the tags for each part of the message separately. For instance, the adversary can send in message $part_{one} = m_1 \parallel m_2$ and generate the corresponding tag $tag_{one} = F_k(\langle 1 \rangle \parallel m_1) \oplus F_k(\langle 2 \rangle \parallel m_2)$. Then the adversary can run a query with message $part_{two} = m_2 \parallel m_3$ and output tag $tag_{two} = F_k(\langle 1 \rangle \parallel m_2) \oplus F_k(\langle 2 \rangle \parallel m_3)$. The adversary can continuously run these queries on two message parts at a time until the adversary gets to message $part_{end} = m_{\ell-1} \parallel m_{\ell}$ where tag is $tag_{end} = F_k(\langle 1 \rangle \parallel m_{\ell-1}) \oplus F_k(\langle 2 \rangle \parallel m_{\ell})$. Here we know that since we are performing XORs on each message $n/2$ -bit part, we know that $part_{one} \oplus part_{two} \oplus \dots \oplus part_{end} = m_1 \parallel m_\ell$. Therefore we also have that each tag $tag_{one} \oplus tag_{two} \oplus \dots \oplus tag_{end} = F_k(\langle 1 \rangle \parallel m_1) \oplus F_k(\langle 2 \rangle \parallel m_{\ell})$. So if A simply runs queries to find the sub tags of each consecutive n bits and outputs a tag combining the first and last $n/2$ -bits of the message input, the output of the tag generation algorithm is still a valid tag for the original message even for a fixed-length message, which shows that this MAC is insecure.

Exercise 2

(15 points) Let $F: \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a PRF. Consider the following MAC = (Gen, Tag, Ver) given by

- Gen(1^n): outputs a uniformly random key $k \leftarrow \{0, 1\}^n$.
- Tag(k, m): given a message $m = m_1 \parallel m_2 \in \{0, 1\}^{2n}$, where $|m_1| = |m_2| = n$, output the tag $t := F_k(m_1) \parallel F_k(F_k(m_2))$.
- Ver(k, m, t): check whether Tag(k, m) = t .

Is this MAC secure (existential unforgeability under chosen message attack)? Prove your answer either by giving a formal reduction to F being a PRF or by constructing a forger with non-negligible advantage in the security game against MAC.

Solution

This MAC is secure (existential unforgeability under chosen message attack). The high-level is that a potential attacker does not know the key k and the outputs of the calls to F_k are indistinguishable from any other input and there is no forge attack that can switch m_1 and m_2 in order to produce a valid concatenated tag with the tag generation algorithm without $\text{negl}(n)$ probability because of PRF indistinguishability. We can give a formal reduction of F being a valid PRF. We can consider an adversary A that has the ability to forge another tag with non-negl probability that is valid for a message. If this adversary exists, there can be an adversary B that can distinguish F_k from a random function from a random function family, therefore breaking pseudorandomness for F_k . We can define B as such: B has two worlds (world 0: a challenger with a random function from a random function family or world 1: a challenger with F_k). Then B simulates the worlds based on what A queries with message m . If the challenger is world 0, B generates the tags with the F_k algorithm using A . If the world is 1, B uses a random output. Therefore if A produces a valid tag in a world, B knows that the world is world 0.

This reduction only holds if it is possible for A to forge a new tag that is valid for message m . Since A does not know k , the tag cannot be computed in any case because the tag generation algorithm relies on F_k , there is no way without $\text{negl}(n)$ probability that A can output a valid tag for m . Therefore this MAC scheme is secure and F is a PRF because it is indistinguishable from a random function in a random function family by the reduction.

Exercise 3

(20 points) For each of the following hash functions H , determine whether it is *necessarily* collision resistant. If it is, describe and analyze an appropriate reduction. If it's not, describe and analyze a feasible attacker against the collision resistance of H .

- (a) $H^s(x) = H_1^s(x) \oplus H_2^s(x)$, where $\{H_1^s\}, \{H_2^s\}$ are both collision-resistant hash function families.
- (b) $H^s(x) = \tilde{H}^s(\tilde{H}^s(x))$, where $\{\tilde{H}^s\}$ is a collision-resistant hash function family (that can support arbitrary length inputs).

Solution

a) This hash function is not necessarily collision-resistant. We can define an attacker that inputs two different inputs x and y into the hash function. We know that there is a collision if $H^s(x) = H^s(y)$. Using the function definition, this means that in this case

$H_1^s(x) \oplus H_2^s(x) = H_1^s(y) \oplus H_2^s(y)$ where these are from collision-resistant hash function families. We can also write this as $H_1^s(x) \oplus H_1^s(y) = H_2^s(x) \oplus H_2^s(y)$ due to the properties of $\oplus$. Although these are two collision-resistant hash functions for ONE instance of calling the function, the XOR of the two outputs could be the same if the attacker were to run many queries of x and y such that the XOR of the two outputs from each hash function computing hashes for x and y can be the same because although the hash functions enforce collision resistance, the XOR does not so after A runs many many queries of x and y on the hash function, it is possible that for one pair of x and y the hash is computed to be the same.

b) This is a necessarily collision resistant hash function. We can write the reduction by proving the opposite case: We have to prove that if there exists an attacker that finds a collision in $\tilde{H}^s$ when $\tilde{H}^s$ has equal security to H^s , then this attacker can break collision resistance of H^s .

If there exists an adversary A that can find two different inputs x and y such that $H^s(x) = H^s(y)$, then it holds by the definition of the hash function H^s that $\tilde{H}^s(\tilde{H}^s(x)) = \tilde{H}^s(\tilde{H}^s(y))$. Knowing this, if we say that $\tilde{H}^s$ is collision resistant, then $\tilde{H}^s(x) \neq \tilde{H}^s(y)$ because x and y are not equal. Therefore we can call the output of $\tilde{H}^s(x) = z_1$ and $\tilde{H}^s(y) = z_2$. Now the adversary needs a z_1 and z_2 such that $\tilde{H}^s(z_1) = \tilde{H}^s(z_2)$. The attacker cannot find a collision for $\tilde{H}^s$ for two different inputs without $\text{negl}(n)$ probability because of the definition of a collision resistant hash function. So it must hold that $\tilde{H}^s(z_1) \neq \tilde{H}^s(z_2)$. Therefore there is no feasible adversary by this proof that can find a collision in $\tilde{H}^s$ without breaking the collision resistance of a collision-resistant function.

Exercise 4

(15 points) Recall the following definitions for MAC schemes.

Definition 1 (Existentially Unforgeable under Chosen Message). A MAC scheme $(\text{Gen}, \text{Mac}, \text{Vrfy})$ is existentially unforgeable under a chosen-message attack if and only for every PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $n \in \mathbb{N}$, $\Pr[\text{Expt}_{\text{EU}}^{\mathcal{A}}(1^n) = 1] \leq \text{negl}(n)$.

$\text{Expt}_{\text{EU}}^{\mathcal{A}}(1^n)$.

- $k \xleftarrow{\$} \text{Gen}(1^n)$;
- $(m, t) \leftarrow \mathcal{A}^{\text{MAC}_k(\cdot)}(1^n)$; (The adversary has oracle access to the MAC function parameterized with k to get tags of messages)

- Output 1 if $\text{Vrfy}(k, m, t) = 1$ and m not queried by $\mathcal{A}$.

In words, the adversary is able to make queries to get tags for a polynomial amount of messages and then make a message tag pair itself that is not of a message previously queried.

Definition 2 (Strongly Existentially Unforgeable under Chosen Message). A MAC scheme $(\text{Gen}, \text{Mac}, \text{Vrfy})$ is strongly existentially unforgeable under a chosen-message attack if and only for every PPT adversary $\mathcal{A}$, there is a negligible function $\text{negl}(\cdot)$ such that for all $n \in \mathbb{N}$, $\Pr[\text{Expt}_{\text{SEU}}^{\mathcal{A}}(1^n) = 1] \leq \text{negl}(n)$.

$\text{Expt}_{\text{SEU}}^{\mathcal{A}}(1^n)$.

- $k \xleftarrow{\$} \text{Gen}(1^n)$;
- $(m, t) \leftarrow \mathcal{A}^{\text{MAC}_k(\cdot)}(1^n)$; (The adversary has oracle access to the MAC function parameterized with k to get tags of messages)
- Output 1 if $\text{Vrfy}(k, m, t) = 1$ and (m, t) is not a pair seen in the set of queries made by $\mathcal{A}$.

In words, the adversary is able to make queries to get tags for a polynomial amount of messages and then make a message tag pair itself that **can be a message previously queried, but the tag cannot be a tag associated with that message from a previous query.**

Prove that if a MAC scheme $(\text{Gen}, \text{Mac}, \text{Vrfy})$ is strongly existentially unforgeable under a chosen-message attack, then it is existentially unforgeable under a chosen-message attack.

Solution

We can prove this by constructing a MAC scheme that is existentially unforgeable under a CMA from a MAC scheme that is strongly existentially unforgeable under CMA. We have a MAC scheme that is strongly existentially unforgeable, therefore we define it as

$k \xleftarrow{\$} \text{Gen}(1^n)$

$(m, t) \leftarrow \mathcal{A}^{\text{MAC}_k(\cdot)}(1^n)$

Output 1 if $\text{Vrfy}(k, m, t) = 1$ and m not queried by $\mathcal{A}$

Under this definition, suppose we have an adversary A that breaks strongly existentially unforgeable security of the MAC scheme with non-negl probability. That would mean that A is able to produce a $(m_A, t_A) \leftarrow \mathcal{A}^{\text{MAC}_k(\cdot)}(1^n)$ such that m_A is a message that was not queried to the oracle (a new message) and $\text{Vrfy}(k, m_A, t_A) = 1$ so the tag for the adversary's message is valid for that message. Now supposed we have an adversary B such that B uses A to break existentially unforgeable MAC schemes under

a CMA.

We have a MAC scheme that is existentially unforgeable, therefore we define it as

$k \xleftarrow{\$} \text{Gen}(1^n)$

$(m, t) \leftarrow \mathcal{B}^{\text{MAC}_k(\cdot)}(1^n)$

Output 1 if $\text{Vrfy}(k, m, t) = 1$ and (m, t) is not a pair seen in the set of queries made by $\mathcal{B}$

Here we have a case where if we send $(m_A, t_A) \leftarrow \mathcal{B}^{\text{MAC}_k(\cdot)}(1^n)$ to the oracle of the MAC in the new security game, then it can be that m_A is not queried to the MAC oracle therefore the output of A is the output of B and the adversary B wins the security game because it is the same scenario as the first game with A . If m_A is queried to the MAC oracle, then B needs to produce a valid tag such that $\text{Vrfy}(k, m_A, t_A) = 1$ but also needs to ensure that (m_A, t_A) is not the same as (m_A, t_B) where t_B is the tag output by the oracle in the query. We know this can never be true though since the definition of B holds that the adversary can only succeed if the message it sends to the game and wins on is not previously queried. Therefore it holds that if A can win the security game on strongly existentially unforgeable MAC under CMA, then B which is constructed from A can win the security game on existentially unforgeable MAC under CMA by computing a different valid tag than the one seen in A 's game for the same message. Therefore by contrapositive we know that if a MAC scheme is existentially unforgeable, then it must be strongly existentially unforgeable under CMA because the latter is a stronger security than the former and breaking stronger existentially unforgeability breaks existentially unforgeability. Therefore in the existentially unforgeability game B 's success has to be $\text{negl}(n)$ probability because B is constructed by A and by definition A has $\text{negl}(n)$ probability of success.

Exercise 5

(30 points) Let $H: \{0, 1\}^\ell \rightarrow \{0, 1\}^n$ be a collision-resistant hash function, where $\ell > n$, and $F: \mathcal{K} \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ be a PRF. Show that the function $F': \mathcal{K} \times \{0, 1\}^\ell \rightarrow \{0, 1\}^n$ given by $F'_k(x) := F_k(H(x))$ is a PRF. In practice, this construction can produce a PRF with arbitrary-length inputs.

Hint: Consider the similarities and differences between this construction and the hash-and-tag construction that we analyzed in lecture.

Solution

Background information from lecture

We know that we have a function $F': \mathcal{K} \times \{0, 1\}^\ell \rightarrow \{0, 1\}^n$ given by $F'_k(x) := F_k(H(x))$. Additionally, we know a function is a PRF if for any PPT A , the adversary cannot dis-

tinguish between $F'_k(x) := F_k(H(x))$ for a random k and a random function $R(x)$ that also maps $R': \{0, 1\}^\ell \rightarrow \{0, 1\}^n$. The adversary A cannot distinguish between the random function from a random function family and a PRF without $\text{negl}(n)$ probability. Additionally we know that the hash function is a collision-resistant hash function, so we know that for any x input given by A , $H(x)$ is not the same for a different input of x without $\text{negl}(n)$ probability as well. Therefore the adversary should not be able to find an $F_k(H(x)) = F_k(H(y))$ for some different x and y inputs and therefore cannot find an $F_k(x_{new}) = F_k(y_{new})$ for some generated hashes. Additionally the adversary cannot distinguish between the two different $F_k(x_{new}), F_k(y_{new})$ without $\text{negl}(n)$ probability because the PRF should be indistinguishable in output from a random function output. In order to show all of this we can first prove that F'_k is a pseudorandom function.

Proof

We know that $H(x)$ maps $\{0, 1\}^\ell$ to a unique $\{0, 1\}^n$ with $\text{negl}(n)$ probability of it not being unique. Therefore any two different inputs x and y will produce $H(x) \neq H(y)$. Therefore the input to the PRF F_k must be unique except for $\text{negl}(n)$ probability for each unique input x . Next, we know F_k is a PRF and therefore any output is indistinguishable from a completely random value as long as k is unknown. Therefore if there is an adversary A , they must distinguish F'_k from a random function $R(x)$. Since $H(x)$ is fixed-length n in the output, the adversary cannot distinguish by length. Since we have proven all outputs of $H(x)$ given that x is different are unique and of n length, the random function R outputs a random n bits length string, which corresponds to the size and variability of $H(x)$ and therefore preserves $F_k(H(x))$'s indistinguishability. Therefore it must hold that $F'_k(x) := F_k(H(x))$ is indistinguishable from a random function of output n bits length and therefore is a PRF by definition. We can also show that F'_k is a PRF through reduction. If there is an adversary A that can distinguish $F'_k(x) := F_k(H(x))$ from a random function with non- negl probability, then there also exists an adversary B that can distinguish F_k and provide a collision for H . B can simulate A by running queries on F'_k . For each input x that B queries, B can compute $H(x)$ from x and send the output to a query for F_k . Here we run into a contradiction since we know that F_k is pseudorandom from our above proof as long as H is collision-resistant. Here the adversary must run two queries that return the same $H(x)$ for two unique x inputs for the pseudorandomness of F_k to break, which is only possible if $H(x)$ is not collision-resistant. Therefore if A can distinguish between F'_k and $R(x)$, B must distinguish between F_k and $R(x)$ or find a collision in H , so A cannot succeed in distinguishing and therefore B cannot either. So the probability of A distinguishing between F'_k and $R(x)$ is $\text{negl}(n)$ and therefore F'_k is a PRF under any unique x input.